

Markdown analyzer

Aplikace pro analýzu markdownu STePSEnHECs-PaCt

Studenti: Bc. Trong Quoc Huy Dinh, Bc. Břetislav Milota,
Bc. Martin Reich, Bc. Eliška Toncarová
Osobní čísla: A25N0040P, A25N0052P (asi dodat ostatní)
Emaily: qhdt@students.zcu.cz, bmilota@students.zcu.cz (asi další emaily)
Datum: 16. 5. 2026

Obsah

1	Úvod	2
2	Použité technologie	2
3	Architektura aplikace	2
3.1	Diagramy	2
3.2	Moduly	5
3.2.1	Core	5
3.2.2	Ui	7
3.2.3	Utils	7
4	Validace markdown souborů	8
5	Uživatelské rozhraní	8
5.1	Hlavní rozvržení okna	8
5.2	Pracovní pohled Markdown (Markdown Scene)	8
5.3	Pracovní pohled Git (Git Scene)	9
6	Testování	9
6.1	Jednotkové testy	9
6.2	Testovací scénáře	9
7	Další možná rozšíření	17
8	Návod k buildu	18
8.1	Požadavky	18
8.2	Koncový uživatel: spuštění programu	19
8.3	Sestavení spustitelného souboru (build skripty)	19
8.4	Vývojářské prostředí a závislosti	19
8.5	Poznámky	20

1 Úvod

Markdown analyzer je desktopová aplikace určená k analýze a editaci markdown souborů podle STePSEnHECs-PaCt katalogu vzorů. Aplikace zároveň disponuje Git klientem pro jednodušší ukládání provedených změn na markdown souborech. Aplikace je spustitelná na operačních systémech Windows a MacOS.

2 Použité technologie

Tato sekce obsahuje přehled hlavních technologií, knihoven a nástrojů, které byly použity při návrhu a vývoji aplikace.

Programovací jazyk:

- **Python 3.10**

Použité knihovny:

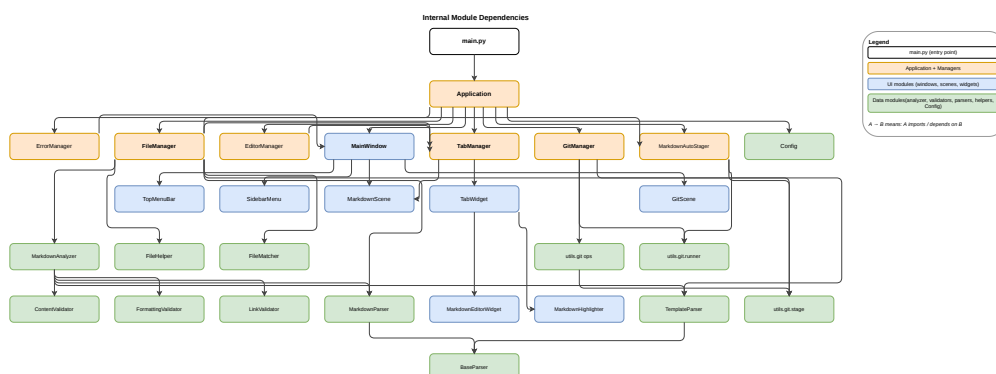
- **PyQt6:** Framework pro tvorbu multiplatformního desktopového rozhraní.
- **Python-Markdown (markdown):** Standardní knihovna pro parsování zdrojového textu a jeho konverzi do HTML pro účely živého náhledu (Live Preview).
- **PyMdown Extensions (pymdownx):** Sada rozšíření pro parser Markdownu, která aplikaci dodává podporu pro moderní syntaxi.
- **Git:** Nativní podpora pro správu verzí v lokálních repozitářích.

3 Architektura aplikace

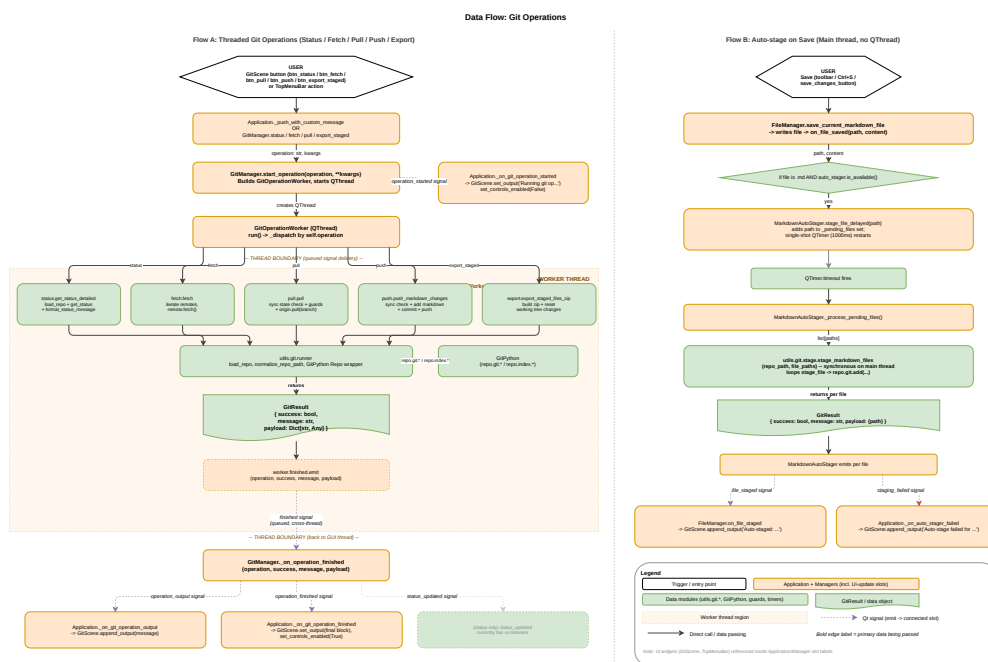
Tato sekce popisuje architekturu aplikace, rozdělení do hlavních modulů a tok dat při zpracování markdown souborů či komunikaci s verzovacím systémem Git.

3.1 Diagramy

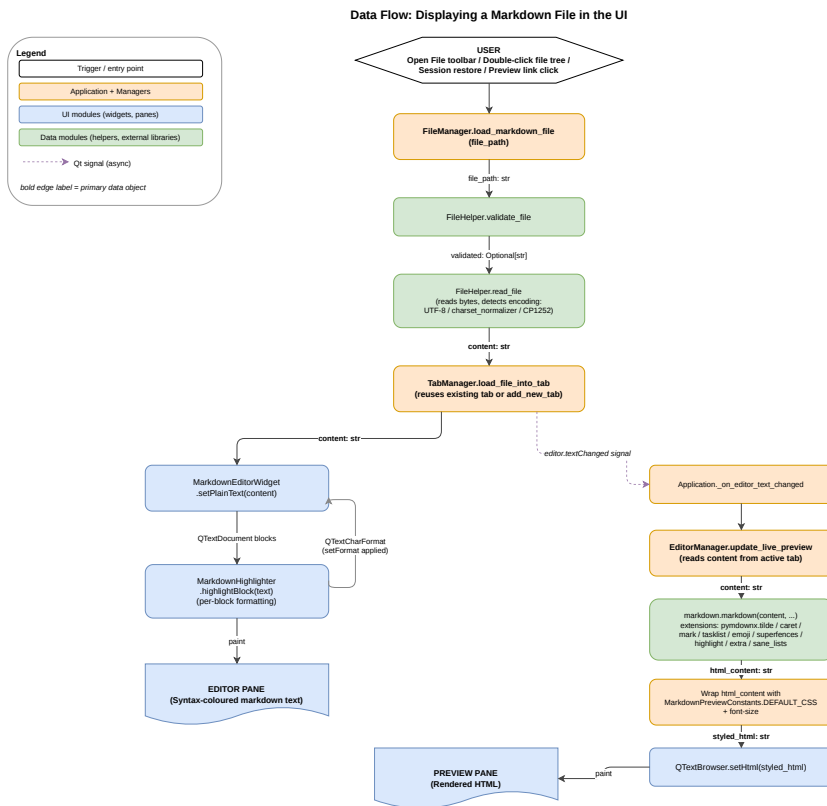
Následující obrázky shrnují závislosti mezi moduly a tok dat při klíčových činnostech aplikace. Diagramy zobrazení a analýzy zahrnují i vrstvy editoru a analyzátoru, aby byl vidět kontext práce se soubory v celém Markdown pohledu.



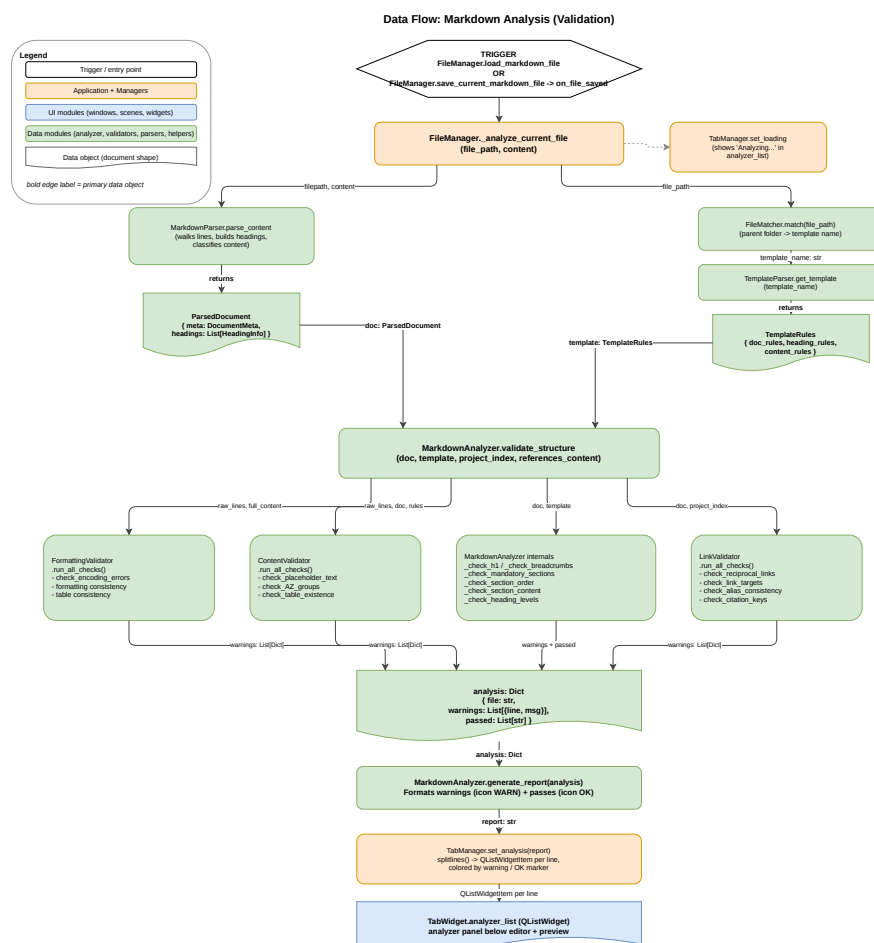
Obrázek 1: Závislosti hlavních modulů aplikace (importy a vrstvy mezi vstupním bodem, aplikací, manažery, uživatelským rozhraním a pomocnými knihovnami).



Obrázek 2: Tok dat při Git operacích: akce uživatele v Git Scene nebo v menu, orchestrace v GitManager, provedení příkazů ve vlákně GitOperationWorker pomocí balíku utils/git a GitPython, zpětná vazba do konzole rozhraní.



Obrázek 3: Tok dat při zobrazení markdown souboru: **FileManager** využívá **FileHelper** k validaci a načtení obsahu, **TabManager** obsluhuje záložky a editor; živý náhled vzniká převodem markdownu na HTML a zobrazením v náhledovém widgetu.



Obrázek 4: Tok dat při analýze a validaci struktury markdownu po načtení nebo uložení: od FileManager přes parsování dokumentu, výběr šablony až po validátory a výpis výsledků v panelu analyzátoru.

3.2 Moduly

Tato sekce popisuje logické a adresářové členění zdrojového kódu aplikace. Architektura projektu je navržena modulárně, s jasným oddělením uživatelského rozhraní (prezentační vrstvy) od aplikační logiky a pomocných nástrojů.

3.2.1 Core

Modul core tvoří samotné jádro celé aplikace. Je zde soustředěna veškerá business logika, správa stavu, propojování komponent a analytické nástroje.

- **analyzer** – DOPLNIT
- **managers** – Tento balíček slouží jako hlavní řídicí (controller) vrstva aplikace. Třídy v této složce oddělují aplikační logiku od samotného uživatelského rozhraní (UI). Jejich úkolem je koordinace procesů, zpracování událostí z UI a komunikace mezi vizuálními komponentami a nízkoúrovňovými nástroji (jako je analyzátor, Git nebo souborový systém).
 - **EditorManager** (`core/managers/editor_manager.py`):

`EditorManager` má na starosti řízení samotného textového editoru a jeho doplňkových vizuálních panelů. Zpracovává změny stavu přepínačů pro zobrazení živého náhledu a výstupu analyzátoru. Jeho klíčovou funkcí je dynamické převádění surového markdownu do formátovaného HTML kódu pomocí knihovny `markdown` a široké sady rozšíření `pymdownx`. Přebírá obsah z aktuální záložky (`TabManager`) a zajišťuje jeho okamžitou vizuální reprezentaci v panelu `Live Preview`.
 - **ErrorManager** (`core/managers/error_manager.py`):

`ErrorManager` centralizuje zobrazení vyskakovacích dialogů a chybových hlášení napříč celou aplikací. Pomocí třídních (statických) metod poskytuje jednotné rozhraní pro vyvolání informačních zpráv, varování, kritických chyb či interaktivních dotazů (jako je volba uložení/zahození změn při zavírání aplikace). Všechna tato upozornění interně realizuje prostřednictvím standardní UI komponenty `QMessageBox`.
 - **FileManager** (`core/managers/file_manager.py`):

`FileManager` koordinuje práci s dokumenty v uživatelském rozhraní: načítání šablon z adresáře, otevírání a ukládání markdown souborů ve spolupráci se `TabManager`, spouštění analýzy přes `MarkdownAnalyzer` a obsluhu průzkumníka souborů. Pro čtení, zápis a dialogy deleguje na `FileHelper`. Propojení s oknem, menu a záložkami je navázáno z třídy `Application` (`core/application.py`), včetně zpracování kliknutí na odkazy v náhledu.
 - **GitManager** (`core/managers/git_manager.py`):

`GitManager` slouží jako správce pro operace s verzovacím systémem Git napojený na uživatelské rozhraní. Aby nedocházelo k zamrzání aplikace při síťových nebo diskových operacích, deleguje spouštění příkazů (např. `push`, `pull`, `commit`) na asynchronní pracovní vlákno `GitOperationWorker`. O výsledcích, průběhu a chybách následně informuje UI pomocí systému událostí (signálů).

– **TabManager** (`core/managers/tab_manager.py`):

TabManager spravuje životní cyklus záložek v hlavním pracovním prostoru (**MarkdownScene**). Pomocí pomocné datové třídy **TabState** udržuje kontext o každé otevřené záložce, včetně cesty k načtenému souboru a indikace neuložených změn (tzv. *dirty state*, reprezentovaný hvězdičkou v názvu). Zajišťuje logiku otevírání, zavírání a přepínání záložek, předává textové změny k dalšímu zpracování a rovněž propisuje formátované výsledky z analyzátoru do odpovídajícího UI panelu v konkrétní záložce.

- **application.py** – Třída **Application** představuje centrální bod celé aplikace. Slouží k inicializaci všech klíčových komponent (hlavní okno, analyzátor, manažeři, pomocné nástroje) a k propojení uživatelského rozhraní s aplikační logikou. Zajišťuje propojení událostí z UI (jako jsou kliknutí na tlačítka, výběr z menu či změny zaškrťovacích políček) s konkrétními akcemi v jednotlivých manažerech. Dále řídí životní cyklus aplikace, což zahrnuje aplikaci grafického motivu (QSS), spuštění hlavní smyčky událostí, bezpečné načítání předchozího stavu při startu (např. pozice okna, otevřené záložky, poslední adresář) a jeho ukládání při ukončování aplikace.

3.2.2 Ui

Modul `ui` obsahuje definici uživatelského rozhraní aplikace:

- **assets** – aktiva aplikace (ikony a QSS vzhledy.)
- **components** – ui komponenty použité v jednotlivých scénách.
- **scenes** – scény aplikace.
- **main_window.py** – definice hlavního okna aplikace.

3.2.3 Utils

Modul `utils` obsahuje pomocné nástroje aplikace.

- **FileHelper** (`utils/file_helper.py`):

Třída **FileHelper** soustředí nízkourovňové operace se soubory a systémovými dialogy PyQt6. Poskytuje výběr jednoho či více markdown souborů a pracovního adresáře (**QFileDialog**), validaci existence souboru, typu a oprávnění ke čtení, čtení a zápis s detekcí kódování (UTF-8, případně knihovna `charset_normalizer`, záložní CP1252) a zapamatováním kódování pro konzistentní uložení. Dále

vyhledává markdown soubory v adresáři (s možností rekurze), hlídá rozumnou velikost obsahu podle `FileConstants` a pomáhá při rozlišení relativních odkazů z náhledu (`resolve_relative_markdown_link`).

- Git operace (`GitManager`, balík `utils/git`):

Správu příkazů Git zajišťuje `GitManager` (`core/managers/git_manager.py`), který ve vlastním vlákne spouští `GitOperationWorker` odvozený od `QThread`. V jeden okamžik může běžet nejvýše jedna operace; další požadavky jsou odmítnuty s hláškou pro uživatele. Pracovník podle typu operace volá funkce z balíku `utils/git` (stav repozitáře, `fetch`, `pull`, `push` včetně `commit` zprávy, `staging`, export staged změn do archivu apod.), přičemž práce s repozitářem probíhá přes knihovnu `GitPython`. Výsledky jsou předávány jako `GitResult`. Nebezpečná akce exportu vyžaduje potvrzení dialogem o možné ztrátě dat. Signály manažeru aktualizují `Git Scene` (konzole, povolení ovládacích prvků); konkrétní propojení je v `Application`.

4 Validace markdown souborů

DOPLNIT

5 Uživatelské rozhraní

Aplikace plně využívá vlastní tmavý motiv (Dark Mode) a vektorové ikony. Rozhraní je rozděleno do dvou hlavních pracovních pohledů (scén), mezi kterými lze přepínat.

5.1 Hlavní rozvržení okna

- **Postranní panel (Sidebar):** Úzký svislý panel na levé straně okna, který slouží k rychlému přepínání mezi editorem Markdownu a rozhraním pro správu Gitu.
- **Horní menu a lišta (Toolbar):** Poskytuje přístup k nejpoužívanějším akcím (otevírání souborů a složek, ukládání, spouštění Git operací) a obsahuje klávesové zkratky.

5.2 Pracovní pohled Markdown (Markdown Scene)

Tento pohled je primárním rozhraním pro úpravu a analýzu markdown souborů.

- **Průzkumník souborů (File Explorer):** Levý postranní panel zobrazující stromovou strukturu načteného pracovního adresáře. Umožňuje rychlou navigaci a otevírání dokumentů dvojitým kliknutím. Pro maximalizaci pracovního prostoru lze panel skrýt.
- **Správce záložek (Tab Manager):** Centrální pracovní plocha umožňující editaci více souborů současně. Záložky vizuálně indikují neuložené změny (hvězdičkou) a chrání uživatele před ztrátou dat při zavření okna.
- **Ovládací panel editoru:** Obsahuje rychlé přepínače (checkboxy) pro zobrazení nebo skrytí živého náhledu a výstupu analyzátoru.
- **Živý náhled (Live Preview):** Panel umístěný napravo od textového editoru, který v reálném čase vykresluje formátovaný HTML výstup (včetně syntaxe kódu, tabulek a odkazů) na základě psaného textu.
- **Výstup analyzátoru (Analyzer Output):** Spodní panel v rámci záložky, který interaktivně vypisuje výsledky strukturální kontroly a validace odkazů. Barvami (červená/žlutá/zelená) odlišuje úspěšné kontroly, varování a kritické chyby.

5.3 Pracovní pohled Git (Git Scene)

Tento pohled slouží k integraci s verzovacím systémem a poskytuje grafické rozhraní pro správu repozitáře.

- **Vstupní pole pro zprávu:** Víceřádkové textové pole, které vynucuje zadání zprávy ke commitu před odesláním změn.
- **Ovládací panel Gitu:** Tlačítka pro spouštění asynchronních git operací (Status, Fetch, Pull, Push, Export Staged).
- **Konzole (Output Console):** Textový výpis umístěný uprostřed, který uživateli poskytuje přímou zpětnou vazbu o probíhajících Git operacích a loguje automatické přidávání souborů (auto-staging) běžící na pozadí.

6 Testování

6.1 Jednotkové testy

6.2 Testovací scénáře

Tato sekce obsahuje hlavní funkční testy pro aplikaci, se zaměřením na uživatelské rozhraní, ošetření chyb a integraci Gitu.

1. Operace s Gitem (Git Operations)

Scénář 1.1: Provedení Git Push se zadanou commit zprávou:

Popis: Ověření, že aplikace provede push, pokud uživatel zadá zprávu ke commitu.

- **DÁNO** uživatel se nachází v pohledu Git (Git scene) a má připravené (staged) změny.
- A textové pole pro commit zprávu obsahuje text.
- **KDYŽ** uživatel klikne na tlačítko "Push".
- **PAK** operace Git push na pozadí se PROVEDE.

Scénář 1.2: Zamezení Git Push s prázdnou commit zprávou:

Popis: Ověření, že aplikace zablokuje push, pokud uživatel zapomene zadat zprávu ke commitu.

- **DÁNO** uživatel se nachází v pohledu Git (Git scene) a má připravené (staged) změny.
- A textové pole pro commit zprávu je zcela prázdné (nebo obsahuje jen mezery).
- **KDYŽ** uživatel klikne na tlačítko "Push".
- **PAK** se zobrazí kritický dialog s titulkem "Missing Commit Message".
- A operace Git push na pozadí se NEPROVEDE.

Scénář 1.3: Export změněných souborů:

Popis: Ověření, že uživatel úspěšně exportuje změněné soubory z pracovního adresáře.

- **DÁNO** uživatel se nachází v pohledu Git.
- **KDYŽ** uživatel klikne na tlačítko "Export Staged".
- **PAK** se zobrazí varovný dialog s titulkem "Warning: Potential Data Loss".
- **KDYŽ** uživatel klikne na "Ano"(Yes).
- **PAK** se dialog zavře a operace exportu se provede.

Scénář 1.4: Zamezení nechtěné ztráty dat při Git Exportu:

Popis: Ověření, že je uživatel varován před exportem, protože tato akce vyčistí pracovní adresář.

- **DÁNO** uživatel se nachází v pohledu Git.
- **KDYŽ** uživatel klikne na tlačítko "Export Staged".
- **PAK** se zobrazí varovný dialog s titulkem "Warning: Potential Data Loss".
- **KDYŽ** uživatel klikne na "Ne"(No).
- **PAK** se dialog zavře a operace exportu se zruší.

2. Ukončení aplikace a správa záložek (Tab Management)

Scénář 2.1: Zavírání aplikace s neuloženými záložkami (Cancel):

Popis: Ověření, že uživatel může přerušit zavírání aplikace, pokud má neuloženou práci.

- **DÁNO** aplikace je otevřená a obsahuje alespoň jednu neuloženou záložku (indikováno hvězdičkou * v názvu záložky).
- **KDYŽ** uživatel klikne na systémové tlačítko pro zavření okna ('X').
- **PAK** se zobrazí varovný dialog s dotazem na Uložení (Save), Zahození (Discard) nebo Zrušení (Cancel).
- **KDYŽ** uživatel klikne na "Cancel".
- **PAK** se dialog zavře a aplikace zůstane otevřená.

Scénář 2.2: Uložení neuložených záložek při zavírání aplikace:

Popis: Ověření, že uživatel může úspěšně uložit svou práci během procesu ukončování aplikace.

- **DÁNO** aplikace je otevřená s neuloženou záložkou.
- **KDYŽ** uživatel klikne na systémové tlačítko pro zavření okna ('X').
- **A** zobrazí se varovný dialog s dotazem na Uložení (Save), Zahození (Discard) nebo Zrušení (Cancel).
- **KDYŽ** uživatel klikne na "Save".
- **PAK** Uloží se daný soubor.
- **A** aplikace se ukončí.

Scénář 2.3: Zahození změn při zavírání aplikace:

Popis: Ověření, že uživatel může úmyslně zahodit změny a vynutit zavření aplikace.

- **DÁNO** aplikace je otevřená s neuloženou záložkou.
- **KDYŽ** uživatel klikne na systémové tlačítko pro zavření okna ('X').
- **A** zobrazí se varovný dialog s dotazem na Uložení (Save), Zahození (Discard) nebo Zrušení (Cancel).
- **KDYŽ** uživatel klikne na "Discard".
- **PAK** jsou neuložené změny ignorovány a aplikace se ukončí.

3. Průzkumník souborů a validace odkazů

Scénář 3.1: Bezpečné zpracování neplatných relativních odkazů:

Popis: Ověření, že kliknutí na nefunkční odkaz v živém náhledu neshodí aplikaci.

- **DÁNO** uživatel si prohlíží markdown soubor v editoru.
- **A** panel živého náhledu (Live Preview) je viditelný.
- **KDYŽ** uživatel v náhledu klikne na relativní odkaz, který směřuje na neexistující soubor (např. `./chybejici_dokument.md`).
- **PAK** se zobrazí kritický dialog s titulkem "Link Error".
- **A** aplikace zůstane stabilní a aktivní.

Scénář 3.2: Bezpečné ošetření smazání souboru mimo aplikaci:

Popis: Ověření, že aplikace bezpečně naskakuje i v případě, že byl dříve otevřený soubor smazán operačním systémem.

- **DÁNO** uživatel má v záložce otevřený soubor `poznamky.md`.
- **A** uživatel ukončí aplikaci.
- **KDYŽ** uživatel smaže soubor `poznamky.md` pomocí průzkumníku souborů ve Windows/macOS.
- **A** uživatel znovu spustí aplikaci.
- **PAK** aplikace úspěšně naskakuje bez pádu.
- **A** záložka pro `poznamky.md` se buď obnoví, nebo se zobrazí čistá, prázdná pracovní plocha bez otevřené záložky.

4. Nastavení a uchování stavu (State Persistence)

Scénář 4.1: Zapamatování rozměrů a rozložení okna:

Popis: Ověření, že aplikace obnoví svou přesnou velikost a pozici na obrazovce.

- **DÁNO** aplikace je spuštěna v okně (není maximalizována).
- **KDYŽ** uživatel změní velikost okna na specifický rozměr a přesune ho do rohu obrazovky.
- **A** uživatel aplikaci zavře a znovu spustí.
- **PAK** se aplikace otevře v úplně stejné pozici a velikosti na obrazovce.

Scénář 4.2: Obnovení dříve otevřených záložek:

Popis: Ověření, že pracovní prostředí uživatele zůstane zachováno přesně tak, jak ho opustil.

- **DÁNO** uživatel má otevřené přesně tři různé markdown soubory ve třech záložkách (A.md, B.md, C.md).
- **A** záložka B.md je aktuálně aktivní/zaměřená.
- **KDYŽ** uživatel aplikaci zavře (a uloží případné změny).
- **A** uživatel aplikaci znovu spustí.
- **PAK** se všechny tři záložky (A.md, B.md, C.md) automaticky obnoví.
- **A** záložka B.md je nastavena jako aktivní/zaměřená.

Scénář 4.3: Obnovení dříve otevřeného pracovního adresáře:

Popis: Ověření, že otevřený pracovní adresář v panelu Průzkumníka zůstane zachován přesně tak, jak ho opustil.

- **DÁNO** uživatel má otevřený pracovní adresář.
- **KDYŽ** uživatel aplikaci zavře.
- **A** uživatel aplikaci znovu spustí.
- **PAK** zobrazí se daný pracovní adresář v panelu Průzkumníka.

5. Živý editor a přepínače rozhraní

Scénář 5.1: Vykreslování živého náhledu v reálném čase:

Popis: Ověření, že živý náhled okamžitě reaguje na změny v editoru.

- **DÁNO** uživatel má otevřený markdown soubor.
- A zaškrtačací pole "Live Preview" je zaškrtnuté a panel náhledu je viditelný.
- **KDYŽ** uživatel do textového editoru napíše *# Nový nadpis*.
- **PAK** se panel živého náhledu okamžitě aktualizuje a zobrazí zformátovaný HTML nadpis *H1*, aniž by bylo nutné soubor uložit.

Scénář 5.2: Přepínání panelů uživatelského rozhraní (konkrétně "Analyzer"):

Popis: Ověření, že UI čistě skrývá a obnovuje panely.

- **DÁNO** uživatel má zobrazený hlavní pohled s markdownem.
- **KDYŽ** uživatel odškrtně pole "Analyzer".
- **PAK** seznam analyzátoru kompletně zmizí z rozložení a uvolní editoru více místa na obrazovce.
- **KDYŽ** uživatel pole "Analyzer" znovu zaškrtně.
- **PAK** se seznam analyzátoru znovu objeví a zachová si svůj předchozí obsah.

Scénář 5.3: Přepínání panelů uživatelského rozhraní (konkrétně "Live Preview"):

Popis: Ověření, že UI čistě skrývá a obnovuje panely.

- **DÁNO** uživatel má zobrazený hlavní pohled s markdownem.
- **KDYŽ** uživatel odškrtně pole "Live Preview".
- **PAK** živý náhled kompletně zmizí z rozložení a uvolní editoru více místa na obrazovce.
- **KDYŽ** uživatel pole "Live Preview" znovu zaškrtně.
- **PAK** se živý náhled znovu objeví s aktualizovaným vzhledem, pokud došlo ke změně obsahu v markdown souboru.

6. Jádru Markdown analyzátoru

Scénář 6.1: Označení chybějících povinných nadpisů:

Popis: Ověření, že analyzátor detekuje porušení struktury šablony.

- **DÁNO** uživatel načte adresář projektu obsahující šablonu (`template_pattern.md`), která vyžaduje nadpis *## Úkoly*.
- **KDYŽ** uživatel otevře soubor odpovídající kategorii této šablony, ale v souboru chybí nadpis *## Úkoly*.

- **PAK** se v panelu Analyzer objeví specifické varování/chyba upozorňující na chybějící požadovanou strukturu.

Scénář 6.2: Analyzátor vymaže chyby po jejich opravě:

Popis: Ověření, že se analyzátor dynamicky aktualizuje, když uživatel chybu opraví.

- **DÁNO** panel Analyzer aktuálně zobrazuje strukturální chybu (např. chybějící povinný nadpis).
- **KDYŽ** uživatel chybějící nadpis do editoru dopíše.
- **A** uživatel klikne na "Save Changes".
- **PAK** se chyba z panelu Analyzer odstraní a zobrazí se stav "Passed" nebo "OK".

7. Integrace Průzkumníka souborů (File Explorer)

Scénář 7.1: Dvojité kliknutí otevře novou záložku: **Popis:** Ověření, že průzkumník správně otevírá nové záložky.

- **DÁNO** uživatel má v panelu Průzkumníka otevřenou složku projektu.
- **A** soubor `sample_pattern.md` NENÍ aktuálně otevřený v žádné záložce.
- **KDYŽ** uživatel dvakrát klikne na `sample_pattern.md` ve stromu Průzkumníka.
- **PAK** se vytvoří nová záložka pro `sample_pattern.md`.
- **A** nová záložka se stane aktivní (zaměřenou).

Scénář 7.2: Dvojité kliknutí na již otevřený soubor jej zaměří:

Popis: Ověření, že průzkumník nevytváří duplicitní záložky pro stejný soubor.

- **DÁNO** uživatel má v panelu Průzkumníka otevřenou složku projektu.
- **A** soubor `sample_pattern.md` je již otevřený v Záložce 1 (záložka má stejný název jako soubor).
- **A** uživatel si aktuálně prohlíží Záložku 2.
- **KDYŽ** uživatel dvakrát klikne na `sample_pattern.md` ve stromu Průzkumníka.
- **PAK** aplikace NEVYTVOŘÍ duplicitní záložku.
- **A** aplikace okamžitě přepne zaměření zpět na Záložku 1.

8. Automatické verzování na pozadí (Background Auto-Stager)

Scénář 8.1: Uložení souboru spustí auto-stager:

Popis: Ověření, že správce Gitu potichu připravuje soubory na pozadí.

- **DÁNO** uživatel má v editoru otevřený sledovaný (tracked) markdown soubor.
- **A** Git repozitář nemá žádné necommitnuté změny.
- **KDYŽ** uživatel provede textovou úpravu a klikne na "Save Changes".
- **PAK** se soubor uloží na disk.
- **A** do konzole v pohledu Git se přidá výstupní zpráva potvrzující, že konkrétní soubor byl úspěšně připraven (auto-staged) na pozadí.

9. Horní menu a klávesové zkratky (Top Menu & Shortcuts)

Scénář 9.1: Otevření souborů a složek přes File menu:

Popis: Ověření, že položky menu a zkratky pro otevírání vyvolají správné systémové dialogy.

- **DÁNO** aplikace je spuštěna a aktivní.
- **KDYŽ** uživatel vybere v horním menu *File* → *Open File* (nebo stiskne *Ctrl+O*).
- **PAK** se otevře systémový dialog pro výběr .md souboru.
- **KDYŽ** uživatel vybere *File* → *Open Folder* (nebo stiskne *Ctrl+F*).
- **PAK** se otevře systémový dialog pro výběr adresáře projektu.

Scénář 9.2: Ukládání pomocí klávesové zkratky:

Popis: Ověření, že klávesová zkratka pro uložení správně ukládá aktivní záložku.

- **DÁNO** uživatel má aktivní záložku s neuloženými změnami (indikováno *).
- **KDYŽ** uživatel vybere *File* → *Save File* (nebo stiskne *Ctrl+S*).
- **PAK** se aktuální soubor uloží na disk.
- **A** indikátor neuložených změn okamžitě zmizí.

Scénář 9.3: Přepínání panelů přes File menu (Toggles):

Popis: Ověření, že menu správně synchronizuje stav UI panelů.

- **DÁNO** uživatel se nachází v hlavním pohledu (Markdown scene).
- **KDYŽ** uživatel vybere *File* → *Open Explorer* (nebo stiskne *Ctrl+E*).
- **PAK** se boční panel průzkumníka souborů skryje (pokud byl zobrazen) nebo zobrazí (pokud byl skryt).
- **A KDYŽ** uživatel stiskne *Ctrl+P* (Live Preview) nebo *Ctrl+A* (Analyzer).
- **PAK** se příslušné panely v editoru zobrazí nebo skryjí a jejich zaškrtnuté políčka (checkboxy) se automaticky zaktualizují.

Scénář 9.4: Spouštění základních Git operací přes menu:

Popis: Ověření, že klávesové zkratky spouští procesy na pozadí správně.

- **DÁNO** uživatel má otevřený repozitář.
- **KDYŽ** uživatel vybere *Git* → *Status* (nebo stiskne *Ctrl+Shift+S*).
- **PAK** se operace spustí.
- **A** i když uživatel není aktuálně v pohledu Gitu (Git scene), po přepnutí do něj uvidí výsledek v konzoli.
- (Poznámka: Tento test se opakuje analogicky pro *Fetch* / *Ctrl+Shift+F* a *Pull* / *Ctrl+Shift+L*).

Scénář 9.5: Ochranné prvky u kritických Git akcí přes menu

Popis: Ověření, že vyvolání riskantních akcí přes menu nebo zkratku neobejde naše ochranné prvky (validaci zprávy a varování o ztrátě dat).

- **DÁNO** textové pole pro commit zprávu je prázdné.
- **KDYŽ** uživatel stiskne *Ctrl+Shift+P* (Push).
- **PAK** se operace zastaví a vyskočí chybový dialog "Missing Commit Message".
- **A KDYŽ** uživatel stiskne *Ctrl+Shift+E* (Export Staged).
- **PAK** vyskočí varovný dialog "Warning: Potential Data Loss" úplně stejně, jako by uživatel kliknul na fyzické tlačítko.

7 Další možná rozšíření

Tato sekce obsahuje seznam dalších možných rozšíření aplikace.

- **Zobrazení struktury souboru** – zobrazení struktury souboru ve formě hierarchie sekcí. Uživatel by měl mít možnost si vybrat, zda se tato hierarchie zobrazí. Proklikem na sekci by se měl editor přesunout na danou sekci v markdown souboru.
- **Vícejazyčná podpora** – podpora více jazyků. V základu podporuje pouze angličtinu.
- **Export do .dot formátu** – export souboru v .dot formátu, který by podle odkazů v markdown souborech zobrazil grafovou strukturu těchto spojů.
- **Vytváření nových souborů** – Aplikace v základu pouze upravuje obsah existujících markdown souborů z pevného disku. V budoucnu by mohla mít možnost vytvářet nové markdown soubory.
- **Přesouvání/přejmenovávání souborů a adresářů** – aplikace v základu pouze otevírá existující soubory z pevného disku. V budoucnu by mohla mít možnost přesouvat a přejmenovávat soubory a adresáře.
- **Vylepšený git commit** – rozdělení řádků markdown souborů do více commitů.

8 Návod k buildu

Tato sekce shrnuje instalaci, sestavení a spuštění aplikace. Podrobnější anglické návody jsou přímo v repozitáři a doplňují ji zejména tyto soubory:

- **application/README.md** – uživatelský manuál (ovládání rozhraní, základní tipy k analýze a Git režimu).
- **application/INSTALLATION_OVERVIEW.md** – rozcestník pro vývojářskou a uživatelskou instalaci.

Koncept odpovídá rozlišení *běžný uživatel* versus *vývojář*: koncový uživatel typicky spouští již sestavený program nebo získá spustitelný soubor z dodaného skriptu; vývojář pracuje se zdrojovým kódem, virtuálním prostředím Pythonu a závislostmi v `application/requirements.txt`. Aplikaci je vhodné spouštět z kořenové složky projektu (obsahuje katalog projektu a metadata Gitu); při špatném pracovním adresáři se program nemusí spustit správně.

8.1 Požadavky

- **Git** – pokud se používají funkce integrace s Git, musí být Git nainstalovaný v systému (git-scm.com/downloads). Ověření: příkaz `git --version`.

- **Python 3.10** – nutný pro vývoj a ruční instalaci závislostí; viz níže. Instalační dokumentace: python.org/downloads.

8.2 Koncový uživatel: spuštění programu

Pokud je k dispozici již sestavený spustitelný soubor, není třeba ručně zakládat virtuální prostředí ani instalovat závislosti z PyPI.

Windows – v adresáři s programem nebo ve složce `dist` (PowerShell akceptuje i zápis s dopřednými lomítky):

```
./markdown_analyzer.exe
./dist/markdown_analyzer.exe
```

Linux / macOS – v adresáři s programem nebo ve složce `dist`:

```
./markdown_analyzer
./dist/markdown_analyzer
chmod +x ./dist/markdown_analyzer # pokud system spusteni odmitne
```

8.3 Sestavení spustitelného souboru (build skripty)

Build skripty leží v **kořeni klonovaného repozitáře** (vedle složky `application/`). Po dokončení sestavení bývá výstup ve složce `dist/`.

Windows (PowerShell) – zápis cesty s dopřednými lomítky je ekvivalentní klasickému tvaru s obrácenými lomítky:

```
cd C:/cesta/k/koreni_projektu
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
./build_script.ps1
./dist/markdown_analyzer.exe
```

Linux / macOS:

```
cd /cesta/k/koreni_projektu
chmod +x build_script.sh # jednou, pokud skript nema pravo ke spusteni
./build_script.sh
./dist/markdown_analyzer
```

8.4 Vývojářské prostředí a závislosti

Pro práci se zdrojovým kódem je doporučeno virtuální prostředí Pythonu a instalace balíčků z `application/requirements.txt`.

1. Ověřit verzi interpretu (Python 3.10.x), např. `python3 --version` nebo na Windows `py --version`.

2. Naklonovat repozitář a přejít do složky aplikace:

```
git clone <HTTPS/SSH>
cd .../application
```

3. Vytvořit a aktivovat virtuální prostředí:

```
python3.10 -m venv venv                # Linux / macOS
py -3.10 -m venv venv                  # Windows, pokud chybí python3.
source venv/bin/activate                # Linux / macOS
# Windows (PowerShell): ./venv/Scripts/Activate.ps1
```

4. Nainstalovat závislosti:

```
pip install -r requirements.txt
# alternativa: python -m pip install -r requirements.txt
```

5. Sestavit balík podle předchozí podsektce z **kořene** repozitáře (`build_script.ps1` / `build_script.sh`), nebo při ladění spouštět modul z aktivovaného prostředí v `application`, např. `python main.py`.

8.5 Poznámky

- Starší verze Pythonu než 3.10 mohou způsobit problémy se závislostmi.
- Na Windows může výchozí politika PowerShellu blokovat spouštění skriptů; řešením je dočasné povolení pro proces (viz výše) nebo nastavení podle dokumentace systému.
- Pokud Git funkce selžou, zkontrolujte instalaci Gitu v systému.